

Learner-State-Aware Generative User Interfaces: A Closed-Loop Control Architecture for Adaptive Learning Environments

Omanand Prashant Swami, Sanjana Shashikant Chavan, K. V. Deshpande,

Pratik Rajendra Patil, Abhishek Ganesh Tirpude

Dept. of Computer Science and Business Systems

JSPM's Rajarshi Shahu College of Engineering, Pune, India

{opswami_csbs, sschavan_csbs, kvdeshpande_comp, prpatil_csbs, agtirpude_csbs}@jspmrscoe.edu.in

Abstract—Intelligent tutoring systems estimate what a learner knows but present a fixed interface, while generative user interfaces (GenUI) synthesize interfaces at runtime without any model of the learner. We propose a closed-loop architecture that couples the two. Bayesian Knowledge Tracing estimates mastery; a PID governor with anti-windup and a filtered derivative converts the mastery deficit into a structural complexity budget; a fast policy selects scaffolding affordances; and a foundation model emits a JSON component tree that must satisfy the budget, with a deterministic fallback when schema or budget checks fail. We evaluate the architecture in silico on 1,000 synthetic learners from three archetypes, replayed through four arms. The closed loop keeps the rendered interface within 0.012 of the governor's budget on average, against 0.131 for unconstrained GenUI and 0.123 for a rule-based baseline, with 3.5 times less jitter than unconstrained GenUI, which matches latent mastery slightly better but skips levels three times as often. Standard BKT saturates after long success runs, so a later streak of errors leaves the interface unchanged; a CUSUM lapse detector cuts overload in a forgetting cohort by 25%. In 300 calls to the real model, 3.4% of outputs were malformed and 2.8% of valid outputs missed the budget, against simulated rates of 3% and 15%. All claims concern algorithmic behavior; no learning gains are claimed.

Index Terms—generative user interfaces, Bayesian knowledge tracing, PID control, cognitive load, adaptive learning, simulation

I. INTRODUCTION

Intelligent tutoring systems (ITS) have long modeled learners precisely. Knowledge tracing maintains a probability that a learner has mastered each knowledge component [1], and tutors built on it have been deployed at scale in mathematics [2], [3]. The interface of such systems, however, is authored in advance: the learner model selects the next problem, not the form in which the problem is presented.

Generative user interfaces take the opposite position. A foundation model produces a task-specific interface for each request [4], including interactive courseware [5]. These systems are open loop: the model does not observe the learner's state, and nothing bounds how complex the generated interface becomes.

Cognitive load theory explains why this matters. The same material can help a novice and hinder an expert (the expertise reversal effect) [6], and guidance should fade as competence

grows [7], [8]. An interface that adapts to the learner therefore needs both a learner model and a mechanism that holds the interface to a complexity appropriate to that model, without the abrupt layout changes that make adaptive interfaces hard to use [9], [10].

This paper makes three contributions:

- 1) A closed-loop architecture in which a BKT mastery estimate drives a PID governor whose output is a structural complexity budget that a generative model must satisfy, with schema validation and a deterministic fallback.
- 2) A computable structural complexity metric over a declarative component tree for linear-equation tasks, built from features linked in prior work to cognitive load and visual complexity.
- 3) An in-silico benchmark of 1,000 synthetic learners across four arms and ablations, including an analysis of BKT saturation after success runs and two remedies.

II. RELATED WORK

A. Learner modeling

Bayesian Knowledge Tracing (BKT) models each knowledge component as a two-state hidden Markov model with prior, learning, guess and slip parameters [1]. Extensions individualize parameters per learner [11], add forgetting [12], or contextualize guess and slip [13], and its fixed points and parameter constraints have been analyzed [14]. Logistic and deep alternatives exist [15]; BKT remains competitive when extended [12]. We use BKT because its state is a calibrated probability that can serve directly as a control variable.

B. Adaptive and generative interfaces

Model-based interface generation [16] adapt presentation to users, but adaptation has costs: unpredictable changes reduce usability [9], [10]. Recent GenUI systems let a large language model produce the interface itself [4], [5]. To our knowledge, none of these systems bounds the generated interface by a learner model.

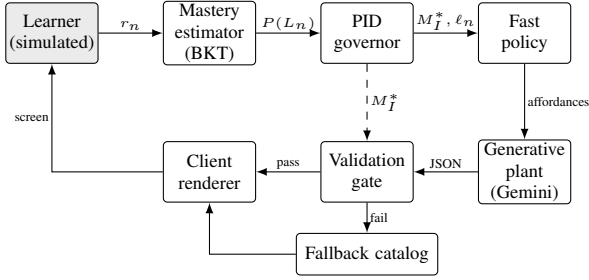


Fig. 1. The closed loop. The governor’s budget M_I^* drives both the policy and the gate (dashed); a tree that fails the schema or budget check is replaced from a precompiled fallback catalog. The learner (shaded) is simulated in this work.

C. Cognitive load and scaffolding

Cognitive load theory [17] attributes intrinsic load to element interactivity [18]. Worked examples reduce load for algebra novices [19] and should fade into problem solving [7, 8]; concrete representations should fade into symbolic ones [20]. For linear equations, the balance model supports early understanding but breaks down with negative quantities [21], and learners hold persistent misconceptions about equality and operating on the unknown [22, 23].

D. Control and constrained generation

PID control with anti-windup and derivative filtering is standard practice [24, 25]. Constrained decoding guarantees well-formed model output [26], though schema-constrained output is not perfectly reliable in practice [27].

III. ARCHITECTURE

The loop has four layers (Fig. 1): a psychometric estimator, a complexity governor, a fast policy, and a generative plant whose output passes a validation gate before a deterministic client renders it.

A. Mastery estimation

After response $r_n \in \{0, 1\}$ the mastery estimate is updated by Bayes’ rule and the learning transition [1]. Writing $p = P(L_n)$, $g = P(G)$, $s = P(S)$ and $\hat{p} = P(L_n | r_n)$,

$$P(L_n | r_n=1) = \frac{p(1-s)}{p(1-s) + (1-p)g}, \quad (1)$$

$$P(L_n | r_n=0) = \frac{ps}{ps + (1-p)(1-g)}, \quad (2)$$

$$P(L_{n+1}) = \hat{p}(1 - P(F)) + (1 - \hat{p})P(T), \quad (3)$$

where $P(F) = 0$ gives standard BKT and $P(F) > 0$ adds forgetting [12]. Default parameters are $P(L_0) = 0.15$, $P(T) = 0.12$, $P(G) = 0.20$, $P(S) = 0.08$. They are design values, not fitted; they satisfy the bounds $P(G) < 0.3$, $P(S) < 0.1$ [1] and the non-degeneracy conditions $P(G) + P(S) < 1$, $P(T) < 1 - P(S)/(1 - P(G))$ [14].

B. Complexity governor

The setpoint is the conventional mastery threshold $P^* = 0.95$ [1, 28]. The tracking error is normalized on each side of the setpoint so that deficit and surplus have equal authority:

$$e_n = \begin{cases} (P^* - P(L_n))/P^* & P(L_n) \leq P^* \\ (P^* - P(L_n))/(1 - P^*) & P(L_n) > P^* \end{cases} \quad (4)$$

so that $e_n \in [-1, 1]$. The control effort is

$$u_n = K_p e_n + K_i S_n + K_d D_n, \quad (5)$$

with the clamped integral $S_n = \text{clamp}(S_{n-1} + e_n, -S_{\max}, S_{\max})$ for anti-windup [25] and the first-order filtered derivative $D_n = \gamma(e_n - e_{n-1}) + (1 - \gamma)D_{n-1}$ [24]. An inverted logistic maps effort to the complexity budget,

$$M_I^* = (1 + e^{ku_n})^{-1}, \quad (6)$$

which is quantized into four levels $\ell_n = \min(4, 1 + \lfloor 4M_I^* \rfloor)$. A hysteresis deadband Δ_h commits a new budget only when it differs from the last committed budget by at least Δ_h .

Gain selection: Classical tuning rules [29, 30] assume an identified plant response. In our benchmark the interface does not alter the simulated learner’s responses (Section IV), so there is no plant response to identify. Instead, K_p , K_i , K_d , γ , $S_{\max}$ and Δ_h were selected by minimizing an ITAE-type tracking cost [31] plus a jitter penalty on a separate tuning cohort; all reported results use a disjoint test cohort. A random search of 200 samples over $K_p \in [0.25, 2.5]$, $K_i \in [0, 0.4]$, $K_d \in [0, 1]$, $\gamma \in [0.05, 1]$, $S_{\max} \in [1, 6]$ and $\Delta_h \in [0.02, 0.15]$ selected $K_p = 0.865$, $K_i = 0.377$, $K_d = 0.672$, $\gamma = 0.398$, $S_{\max} = 1.253$ and $\Delta_h = 0.141$. The cost surface is flat: the selected point scores 0.104 on the tuning cohort against 0.110 for hand-set initial values, which rank 22nd of 201. The logistic slope is $k = \ln 19/(K_p + K_i S_{\max}) = 2.202$, so that the effort range $\pm(K_p + K_i S_{\max})$ maps to $[0.05, 0.95]$.

C. Fast policy

A fast policy maps $(P(L_n), M_I^*, \ell_n, \text{recent errors})$ to discrete affordances: scaffolding mode (worked example, guided scaffold, open symbolic), whether to show the balance-scale manipulative, whether hints are enabled, and an instructional density limit. The policy is deterministic and rule-based, and it is the policy in every reported result. We also evaluated an open-weight decision model [32] zero-shot as a replacement. On a CPU it took a median of 1.32 s per decision (95th percentile 1.46 s) and gave the same decision, a worked example, for all 200 test states, so it is not part of the evaluated system.

D. Generative plant and validation gate

The plant, Gemini 3.7 Flash served through Vertex AI, receives the budget, the policy decision and the equation and emits a JSON component tree. Nodes are drawn from a closed set of six components: CONCRETEBALANCESCALE, WORKEDSOLUTIONSTEP, SCAFFOLDEDOPERATIONPAD,

TABLE I
LEARNER ARCHETYPES (UNIFORM PARAMETER RANGES).

Archetype	$P(L_0)$	$P(T)$	$P(G)$	$P(S)$
Struggling novice	.02–.08	.03–.07	.10–.20	.20–.30
Inconsistent guesser	.05–.20	.10–.20	.15–.40	.08–.30
Fast master	.15–.30	.30–.40	.15–.25	.01–.03

SYMBOLICEQUATIONWORKSPACE, STEPPEDHINTACCORDION and CARTESIANVERIFICATIONPLANE. The client renders these components directly and never executes generated code. Each emission is parsed against the schema; if parsing fails, or if $|M_I - M_I^*| > \varepsilon$ with $\varepsilon = 0.05$, a precompiled fallback tree nearest to the budget is rendered instead. A web client renders the trees for inspection (Fig. 2); the benchmark does not use it.

E. Structural complexity metric

For a tree with ρ interactive elements, abstraction level α and nesting depth δ ,

$$M_I = w_1 \frac{\rho - 1}{7} + w_2 \frac{\alpha - 1}{3} + w_3 \frac{\delta - 1}{3}, \quad (7)$$

with $\rho \in [1, 8]$ and $\alpha, \delta \in [1, 4]$. Each feature has a basis in prior work: element interactivity raises load [18] and display density is a classic complexity measure [33]; the abstraction ladder from balance scale to operation pad to symbolic workspace follows concreteness fading [20]; and deeper structures cost users more [34]. Placing the Cartesian plane at the top of the ladder is our design choice. With no data to fit weights, we use equal weights $w_i = 1/3$, which are robust in that setting [35], and test sensitivity to the weights (Section V). M_I is a structural proxy; it has not been validated against measured cognitive load.

IV. METHOD

A. Synthetic learners

Simulated learners are an established way to test instructional designs before human studies [36], [37], and BKT-generated data have been used to evaluate mastery assessment [38]. Each learner is a two-state HMM whose parameters are drawn uniformly from archetype ranges (Table I). The ranges for the novice and guesser archetypes exceed the tighter bounds of [1] by design to represent noisy learners, while respecting the non-degeneracy conditions of [14]. The cohort has 1,000 learners (300/400/300) with 40 items each. Responses are sampled once per learner and replayed unchanged through every arm (paired design). The interface therefore does not influence the simulated responses: the benchmark measures controller behavior, not learning.

B. Arms

(1) *Static*: a fixed level-2 interface. (2) *Unconstrained GenUI*: the plant chooses complexity from a noisy reading of mastery with no budget check. (3) *Rule-based*: step up after three consecutive correct responses and down after two

consecutive errors, an up-down rule in the spirit of [39]. (4) *Closed loop*: the full architecture. Ablations remove the derivative filter, remove anti-windup, and add a one-level slew limit.

C. Plant model in the benchmark

The benchmark replaces the foundation model with a surrogate that compiles the closest admissible tree and injects faults: with probability 0.15 it drifts off budget, and with probability 0.03 it emits malformed output (truncated JSON, unknown component, out-of-range field, depth overflow, injected field). These rates were set before the real model was measured. The unconstrained arm uses the same surrogate without the budget check.

To check them, we sent 300 requests to the real model (JSON output mode without a response schema, thinking level low, temperature 0.4, 30 s timeout) with budgets drawn uniformly from $[0.02, 0.98]$ and passed every output through the same gate. Nine calls failed in transport (six HTTP errors, three 30 s timeouts). Of the remaining 291, ten were valid JSON that broke a structural rule (a tree with no interactive element), a content-malformed rate of 3.4%. Of the 281 valid trees, eight (2.8%) missed the budget by more than ε ; the median error was 0.012. Counting all failures, 91% of calls met the budget and 9% would fall back. Compliance was lowest at level 1 (82% within budget) and 93–95% at levels 2–4. Median latency of completed calls was 5.6 s (95th percentile 7.8 s). The surrogate’s 3% malformed rate is therefore close to the measured one, and its 15% drift rate is a stress setting about five times the measured rate.

D. Metrics

Constraint error $\Delta M = |M_I - M_{\text{ref}}^*|$ is measured against the budget the closed-loop governor commits for the same responses, so that all arms share one reference; “within ε ” is the share of steps with $\Delta M \leq 0.05$. Interface jitter is $J = \frac{1}{N-1} \sum_n |M_I(n) - M_I(n-1)|$, and a skip is a change of two or more levels between consecutive items. Jitter operationalizes the stability concern of [9]; the formula is ours. Recovery time is the number of items after a streak of five or more errors until the level returns to its pre-streak value. Fallback rate is the share of steps rendered from the fallback catalog. Because the simulator knows the learner’s latent state, we also report *overload* (level 3 or 4 shown while the skill is not known), *underload* (level 1 or 2 shown while it is known) and an ITAE-weighted mismatch $\sum_n (n+1)m_n / \sum_n (n+1)$ [31], where m_n marks either case. These three measures are available only in simulation.

V. RESULTS

All numbers come from one benchmark run on the test cohort (seed 20261003; tuning seed 20261004), released with the code. Intervals are normal-approximation 95% confidence intervals over the 1,000 learners.

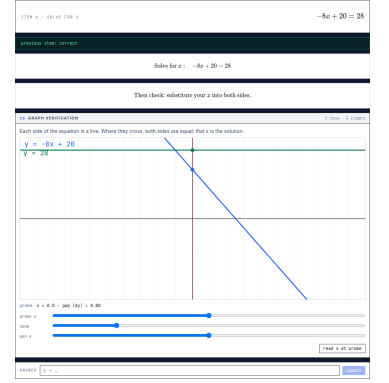


Fig. 2. Two screens rendered by the client. Left: a level-1 tree emitted by Gemini 3.7 Flash for $P(L) = 0.15$, a worked example with the balance-scale manipulative. Right: a level-4 tree from the surrogate plant for $P(L) = 0.999$, a symbolic workspace with graph verification.

TABLE II

MAIN RESULTS AND ABLATIONS ON THE TEST COHORT (1,000 LEARNERS, 40 ITEMS EACH). ΔM AND “WITHIN ε ” ARE MEASURED AGAINST THE CLOSED-LOOP REFERENCE BUDGET; EVERY CLOSED-LOOP VARIANT MEETS ITS OWN BUDGET AT EVERY STEP. OVERLOAD, UNDERLOAD AND ITAE USE THE LATENT LEARNER STATE AND EXIST ONLY IN SIMULATION. FALLBACK RATES FOLLOW FROM THE SURROGATE’S FAULT RATES; ITS DRIFT RATE IS ABOUT FIVE TIMES THAT MEASURED ON THE REAL MODEL.

Arm	ΔM (95% CI)	Within ε	Jitter J	Skips/run	Overload	Underload	ITAE	Fallback
Static	0.426 (0.422–0.430)	0.004	0.000	0.00	0.000	0.800	0.892	–
Unconstrained GenUI	0.131 (0.128–0.134)	0.321	0.123	2.06	0.024	0.054	0.043	0.030
Rule-based	0.123 (0.119–0.128)	0.596	0.025	0.00	0.000	0.245	0.180	–
Closed loop	0.0122 (0.0119–0.0124)	1.000	0.035	0.67	0.002	0.121	0.073	0.154
no derivative filter	0.031 (0.030–0.033)	0.811	0.044	1.19	0.002	0.124	0.076	0.156
no anti-windup	0.107 (0.102–0.112)	0.549	0.024	0.42	0.001	0.217	0.174	0.159
one-level slew limit	0.025 (0.023–0.026)	0.875	0.034	0.02	0.002	0.121	0.073	0.153

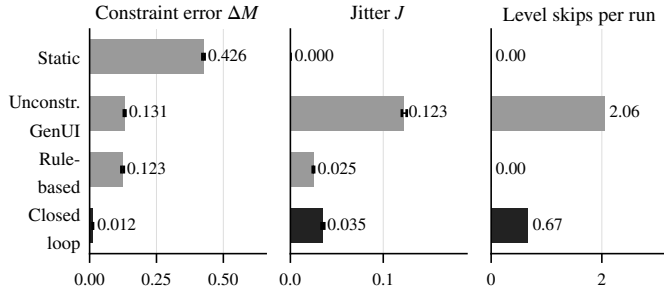


Fig. 3. Main comparison on the test cohort (means with 95% CIs). The closed loop (dark) has the lowest constraint error; the rule-based arm is smoother but underloads (Table II).

A. Main comparison

Table II and Fig. 3 show that the closed loop keeps the rendered interface within 0.012 of its budget on average. Every rendered tree is within ε , but this holds by construction, since the gate substitutes a fallback whenever a tree misses the budget; with the surrogate’s 15% drift rate, 15.4% of steps are fallbacks. Unconstrained GenUI and the rule-based arm track the same reference budget less closely (0.131 and 0.123).

The closed loop’s jitter (0.035) is 3.5 times lower than that of unconstrained GenUI (0.123), which also skips levels 2.06 times per run against 0.67. The rule-based arm is smoother still (0.025, no skips), because it moves one level at a time. Against

the latent learner state, the picture is mixed. Unconstrained GenUI has the lowest ITAE mismatch (0.043 against 0.073) and the least underload (5.4% of steps against 12.1%), but shows an abstract interface to a learner who does not know the skill twelve times as often (2.4% against 0.2%). The rule-based arm underloads on 24.5% of steps and the static arm on 80%. After a streak of five or more errors that lowered the level, the closed loop returned to its pre-streak level within the horizon in 34 of 51 cases, after a median of 5 items; the rule-based arm did so in 9 of 19 cases, after a median of 12.

B. Ablations

Removing the derivative filter raises jitter from 0.035 to 0.044 and nearly doubles level skips (1.19 per run). Removing anti-windup lets the integral grow without bound: underload rises from 12.1% to 21.7%, only 84% of learners reach level 4 against 95%, and median recovery after an error streak grows from 5 to 14 items (only 4 recoveries within the horizon). In a scripted probe of prolonged struggle, the unclamped integral reached 22.9 (clamp 1.25) and the interface never reached level 4. A one-level slew limit removes nearly all skips (0.02 per run) and leaves the ground-truth measures unchanged.

C. Sensitivity

Changing each controller parameter by $\pm 50\%$ raised the test cost (ITAE + J) from 0.108 to at most 0.118. One change, $1.5\Delta_h$, lowered it to 0.104 but more than doubled level skips

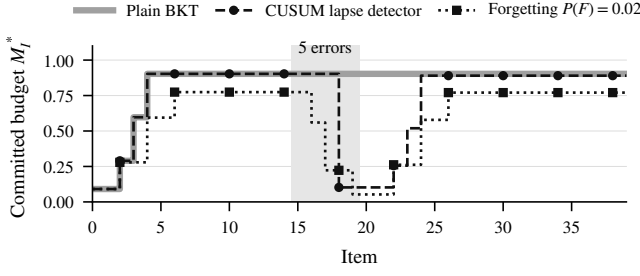


Fig. 4. Scripted probe: 15 correct responses, five errors, then correct again. With plain BKT the estimate is saturated and the budget never moves.

(1.51 per run), so the cost does not capture every property we care about. For the metric weights, the closed loop’s ΔM stays between 0.007 and 0.017 under equal weights, the initial 0.40/0.35/0.25 split and three single-feature-heavy splits (0.6/0.2/0.2). Dropping abstraction or depth roughly doubles it (0.023 and 0.022) while keeping more than 99% of steps within ε . Dropping element density breaks the loop: the fallback rate rises to 84% and even then only 18% of rendered screens are within ε , because the remaining two features cannot reach most budgets (our reading). ITAE mismatch is about 0.072 under every weighting.

D. BKT saturation and lapse handling

Under correct responses the BKT estimate converges to a stable fixed point at one [14]. With the default parameters the estimate exceeds $1 - 10^{-6}$ after ten correct responses and five subsequent errors lower it only to about 0.98; after 24 correct responses it equals 1.0 in double precision and no further evidence can lower it. The interface therefore stays at its highest level through a streak of errors. We compare two remedies: forgetting, $P(F) > 0$ [12], and a CUSUM detector [40] on the log-likelihood ratio between “not known” and the tracker’s prediction, which resets the estimate to 0.5 when the sum exceeds $h = 6.5$. Because it accumulates only when a confident estimate is contradicted, isolated slips are absorbed. The tracker with forgetting uses $P(F) = 0.02$.

In a scripted probe with fifteen correct responses followed by five errors (Fig. 4), the plain closed loop never lowers the interface; the CUSUM variant drops a level 3 items after the streak begins and forgetting after 1. To test this at scale, a second cohort of 1,000 learners can forget the skill with probability 0.02–0.06 per item. There the plain closed loop shows an abstract interface to a learner who does not know the skill on 7.6% of steps. The CUSUM detector lowers this to 5.6% (–25%) at the cost of higher jitter (0.052 against 0.042) and an almost unchanged ITAE mismatch (0.189 against 0.184). Forgetting lowers overload further, to 3.0%, but underloads on 25.3% of steps against 12.8%, raises ITAE to 0.273 and lets only 80% of learners reach level 4. Neither remedy dominates. Adding the slow limit to the CUSUM variant removes its level skips (0.015 per run). Weighting recent responses more heavily [41] is a third option we leave for future work.

VI. DISCUSSION AND LIMITATIONS

The benchmark tests whether the controller does what it is designed to do; it does not test whether learners benefit. Because simulated responses do not depend on the interface, the loop is closed only through the mastery estimate, not through the learner. M_I is a structural proxy that has not been validated against measured load, for which instruments exist [42]. BKT parameters are design values rather than fits to tutor data. Long error streaks may reflect misconceptions rather than slips [23]; the controller does not distinguish these. The benchmark plant is a surrogate. Its malformed rate matches and its drift rate exceeds those measured on the real model, but the real-model measurement covers single calls, not full learner trajectories, and its 5.6 s median latency is too slow for per-item regeneration without caching or prefetching. Finally, a learned fast policy would need fine-tuning on real learner interaction data; training it on simulator data would only imitate the rule policy it is meant to replace.

VII. CONCLUSION

We coupled a learner model to a generative interface through a PID governor, a fast policy and a validation gate, and tested the loop in simulation. The governor’s budget is met at every step, with the gate absorbing plant faults, and the interface changes 3.5 times less than an unconstrained generator while rarely showing abstract screens to simulated learners who have not mastered the skill. Anti-windup and the derivative filter each matter, and a slow limit removes level skips. Standard BKT stops reacting after long success runs; a CUSUM lapse detector restores the reaction at a modest cost in jitter, while forgetting overcorrects. In 300 real calls the generative model met the budget 91% of the time. The next steps are running full trajectories with the real model, a learned fast policy trained on real interaction data, and a study with learners.

CODE AND DATA AVAILABILITY

The engine, simulation harness, benchmark results and the sources of this manuscript are openly available at <https://github.com/omanandswami2005/closed-loop-genui-research-implementation>.

REFERENCES

- [1] A. T. Corbett and J. R. Anderson, “Knowledge tracing: Modeling the acquisition of procedural knowledge,” *User Modeling and User-Adapted Interaction*, vol. 4, no. 4, pp. 253–278, 1995. [Online]. Available: <https://doi.org/10.1007/BF01099821>
- [2] S. Ritter, J. R. Anderson, K. R. Koedinger, and A. Corbett, “Cognitive tutor: Applied research in mathematics education,” *Psychonomic Bulletin & Review*, vol. 14, no. 2, pp. 249–255, 2007. [Online]. Available: <https://doi.org/10.3758/BF03194060>
- [3] K. VanLehn, “The relative effectiveness of human tutoring, intelligent tutoring systems, and other tutoring systems,” *Educational Psychologist*, vol. 46, no. 4, pp. 197–221, 2011.
- [4] Y. Leviathan, D. Valevski, M. Kalman, D. Lumen, E. Segalis, E. Molad, S. Pasternak, V. Natchu, V. Nygaard, V. Srinivasan, J. Manyika, and Y. Matias, “Generative UI: LLMs are effective UI generators,” arXiv:2604.09577, 2026. [Online]. Available: <https://arxiv.org/abs/2604.09577>

- [5] S. Tu, Y. Li, K. Chen, S. Zhang, J. Yu, D. Zhang-Li, L. Hou, J. Li, Y. Zhang, H. Liu *et al.*, “MAIC-UI: Making interactive courseware with generative UI,” arXiv:2604.25806, 2026.
- [6] S. Kalyuga, P. Ayres, P. Chandler, and J. Sweller, “The expertise reversal effect,” *Educational Psychologist*, vol. 38, no. 1, pp. 23–31, 2003. [Online]. Available: https://doi.org/10.1207/S15326985EP3801_4
- [7] A. Renkl and R. K. Atkinson, “Structuring the transition from example study to problem solving in cognitive skill acquisition: A cognitive load perspective,” *Educational Psychologist*, vol. 38, no. 1, pp. 15–22, 2003. [Online]. Available: https://doi.org/10.1207/S15326985EP3801_3
- [8] R. J. C. M. Salden, V. Alevén, R. Schwonke, and A. Renkl, “The expertise reversal effect and worked examples in tutored problem solving,” *Instructional Science*, vol. 38, no. 3, pp. 289–307, 2010. [Online]. Available: <https://doi.org/10.1007/s11251-009-9107-8>
- [9] K. Z. Gajos, K. Everitt, D. S. Tan, M. Czerwinski, and D. S. Weld, “Predictability and accuracy in adaptive user interfaces,” in *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI)*. ACM, 2008, pp. 1271–1274. [Online]. Available: <https://doi.org/10.1145/1357054.1357252>
- [10] L. Findlater and J. McGrenere, “A comparison of static, adaptive, and adaptable menus,” in *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI)*. ACM, 2004, pp. 89–96. [Online]. Available: <https://doi.org/10.1145/985692.985704>
- [11] Z. A. Pardos and N. T. Heffernan, “Modeling individualization in a Bayesian networks implementation of knowledge tracing,” in *User Modeling, Adaptation, and Personalization (UMAP 2010)*, ser. Lecture Notes in Computer Science. Springer, 2010, pp. 255–266. [Online]. Available: https://doi.org/10.1007/978-3-642-13470-8_24
- [12] M. Khajah, R. V. Lindsey, and M. C. Mozer, “How deep is knowledge tracing?” in *Proceedings of the 9th International Conference on Educational Data Mining (EDM 2016)*, 2016. [Online]. Available: <https://arxiv.org/abs/1604.02416>
- [13] R. S. J. d. Baker, A. T. Corbett, and V. Alevén, “More accurate student modeling through contextual estimation of slip and guess probabilities in Bayesian knowledge tracing,” in *Intelligent Tutoring Systems (ITS 2008)*, ser. Lecture Notes in Computer Science. Springer, 2008, pp. 406–415. [Online]. Available: https://doi.org/10.1007/978-3-540-69132-7_44
- [14] B. van de Sande, “Properties of the Bayesian knowledge tracing model,” *Journal of Educational Data Mining*, vol. 5, no. 2, pp. 1–10, 2013. [Online]. Available: <https://jedm.educationaldatamining.org/index.php/JEDM/article/view/35>
- [15] R. Pelánek, “Bayesian knowledge tracing, logistic models, and beyond: An overview of learner modeling techniques,” *User Modeling and User-Adapted Interaction*, vol. 27, no. 3–5, pp. 313–350, 2017. [Online]. Available: <https://doi.org/10.1007/s11257-017-9193-2>
- [16] K. Z. Gajos, D. S. Weld, and J. O. Wobbrock, “Automatically generating personalized user interfaces with Supple,” *Artificial Intelligence*, vol. 174, no. 12–13, pp. 910–950, 2010. [Online]. Available: <https://doi.org/10.1016/j.artint.2010.05.005>
- [17] J. Sweller, “Cognitive load during problem solving: Effects on learning,” *Cognitive Science*, vol. 12, no. 2, pp. 257–285, 1988. [Online]. Available: https://doi.org/10.1207/s15516709cog1202_4
- [18] —, “Element interactivity and intrinsic, extraneous, and germane cognitive load,” *Educational Psychology Review*, vol. 22, no. 2, pp. 123–138, 2010. [Online]. Available: <https://doi.org/10.1007/s10648-010-9128-5>
- [19] J. Sweller and G. A. Cooper, “The use of worked examples as a substitute for problem solving in learning algebra,” *Cognition and Instruction*, vol. 2, no. 1, pp. 59–89, 1985. [Online]. Available: https://doi.org/10.1207/s1532690xci0201_3
- [20] E. R. Fyfe, N. M. McNeil, J. Y. Son, and R. L. Goldstone, “Concreteness fading in mathematics and science instruction: A systematic review,” *Educational Psychology Review*, vol. 26, no. 1, pp. 9–25, 2014. [Online]. Available: <https://doi.org/10.1007/s10648-014-9249-3>
- [21] J. Vlassis, “The balance model: Hindrance or support for the solving of linear equations with one unknown,” *Educational Studies in Mathematics*, vol. 49, no. 3, pp. 341–359, 2002. [Online]. Available: <https://doi.org/10.1023/A:1020229023965>
- [22] C. Kieran, “Concepts associated with the equality symbol,” *Educational Studies in Mathematics*, vol. 12, no. 3, pp. 317–326, 1981. [Online]. Available: <https://doi.org/10.1007/BF00311062>
- [23] J. L. Booth, C. Barbieri, F. Eyer, and E. J. Paré-Blagoev, “Persistent and pernicious errors in algebraic problem solving,” *The Journal of Problem Solving*, vol. 7, no. 1, 2014. [Online]. Available: <https://doi.org/10.7771/1932-6246.1161>
- [24] K. J. Åström and T. Häggglund, *Advanced PID Control*. Research Triangle Park, NC: ISA – The Instrumentation, Systems, and Automation Society, 2006.
- [25] K. J. Åström and L. Rundqwist, “Integrator windup and how to avoid it,” in *1989 American Control Conference*. IEEE, 1989, pp. 1693–1698. [Online]. Available: <https://doi.org/10.23919/ACC.1989.4790464>
- [26] B. T. Willard and R. Louf, “Efficient guided generation for large language models,” arXiv:2307.09702, 2023. [Online]. Available: <https://arxiv.org/abs/2307.09702>
- [27] J. Geng, H. Cooper, M. Moskal, S. Jenkins, J. Berman, N. Ranchin, R. West, E. Horvitz, and H. Nori, “JSONSchemaBench: A rigorous benchmark of structured outputs for language models,” arXiv:2501.10868, 2025. [Online]. Available: <https://arxiv.org/abs/2501.10868>
- [28] J. Zhang, K. Vanacore, R. S. Baker, N. Ch, C. Mills, and O. Henkel, “How much mastery is enough mastery? the relationship between mastery in a lesson and the performance on the subsequent lesson,” in *Proceedings of the 18th International Conference on Educational Data Mining (EDM 2025)*, 2025.
- [29] J. G. Ziegler and N. B. Nichols, “Optimum settings for automatic controllers,” *Transactions of the ASME*, vol. 64, no. 8, pp. 759–765, 1942. [Online]. Available: <https://doi.org/10.1115/1.4019264>
- [30] S. Skogestad, “Simple analytic rules for model reduction and PID controller tuning,” *Journal of Process Control*, vol. 13, no. 4, pp. 291–309, 2003. [Online]. Available: [https://doi.org/10.1016/S0959-1524\(02\)00062-8](https://doi.org/10.1016/S0959-1524(02)00062-8)
- [31] D. Graham and R. C. Lathrop, “The synthesis of “optimum” transient response: Criteria and standard forms,” *Transactions of the American Institute of Electrical Engineers, Part II: Applications and Industry*, 1953. [Online]. Available: <https://doi.org/10.1109/TAI.1953.6371346>
- [32] Convai Innovations, “Laya: Open-weight decision model,” Model card, <https://huggingface.co/convaiinnovations/laya>, 2026.
- [33] T. S. Tullis, “The formatting of alphanumeric displays: A review and analysis,” *Human Factors*, vol. 25, no. 6, pp. 657–682, 1983. [Online]. Available: <https://doi.org/10.1177/001872088302500604>
- [34] J. I. Kiger, “The depth/breadth trade-off in the design of menu-driven user interfaces,” *International Journal of Man-Machine Studies*, vol. 20, no. 2, pp. 201–213, 1984. [Online]. Available: [https://doi.org/10.1016/S0020-7373\(84\)80018-8](https://doi.org/10.1016/S0020-7373(84)80018-8)
- [35] R. M. Dawes, “The robust beauty of improper linear models in decision making,” *American Psychologist*, vol. 34, no. 7, pp. 571–582, 1979. [Online]. Available: <https://doi.org/10.1037/0003-066X.34.7.571>
- [36] K. VanLehn, S. Ohlsson, and R. Nason, “Applications of simulated students: An exploration,” *Journal of Artificial Intelligence in Education*, vol. 5, no. 2, pp. 135–175, 1994.
- [37] T. Käser and G. Alexandron, “Simulated learners in educational technology: A systematic literature review and a Turing-like test,” *International Journal of Artificial Intelligence in Education*, vol. 34, no. 2, pp. 545–585, 2024. [Online]. Available: <https://doi.org/10.1007/s40593-023-00337-2>
- [38] S. E. Fancsali, T. Nixon, and S. Ritter, “Optimal and worst-case performance of mastery learning assessment with Bayesian knowledge tracing,” in *Proceedings of the 6th International Conference on Educational Data Mining (EDM 2013)*, 2013.
- [39] H. Levitt, “Transformed up-down methods in psychoacoustics,” *The Journal of the Acoustical Society of America*, vol. 49, no. 2B, pp. 467–477, 1971. [Online]. Available: <https://doi.org/10.1121/1.1912375>
- [40] E. S. Page, “Continuous inspection schemes,” *Biometrika*, vol. 41, no. 1/2, pp. 100–115, 1954. [Online]. Available: <https://doi.org/10.2307/2333009>
- [41] A. Galyardt and I. Goldin, “Predicting performance during tutoring with models of recent performance,” arXiv:1501.02732, 2015. [Online]. Available: <https://arxiv.org/abs/1501.02732>
- [42] J. Leppink, F. Paas, C. P. M. Van der Vleuten, T. Van Gog, and J. J. G. Van Merriënboer, “Development of an instrument for measuring different types of cognitive load,” *Behavior Research Methods*, vol. 45, no. 4, pp. 1058–1072, 2013. [Online]. Available: <https://doi.org/10.3758/s13428-013-0334-1>